

Compte rendu Framework Symfony TP2:

Le TP que nous avons réalisé porte sur le Framework Symfony.

Un framework est un ensemble d'outils, de bibliothèques et de composants qui fournit une structure déjà définie pour le développement d'applications, afin de faciliter le travail du développeur, d'accélérer la création du projet et d'en améliorer l'organisation.

1/ Introduction Symfony

Symfony nécessite cependant des outils comme Composer pour pouvoir accéder à encore d'autres outils, bibliothèques... qui pourrait être utilisé par le framework lors de l'utilisation.

Bootstraps est utilisé pour le style du site et la présentation il est appelé par ces liens :

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-sR144xILFv4716crS2wB07VP478+LH7qQnuqkuaIAVMlzeNBtE5YBuJzq1LB" crossorigin="anonymous">
<meta name="viewport" content="width=device-width, initial-scale=1">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js" integrity="sha384-FKyoForGlywn9Hj09JcYn3nv7dPV1z7YwJrWcXX/BmVNDxHD25cQbITxI" crossorigin="anonymous"></script>
<link rel="stylesheet" href="{{ asset('assets/css/bootstrap.min.css')}}">
```

Bootstraps utilise des balise div class container pour améliorer le visuel du site.

Symfony s'occupe aussi de créer et d'interagir avec la base de donnée il suffit de mettre dans le .env les références de notre logiciel de création de base de données puis qu'on donne les instructions dans l'invite de commande afin de créer ou modifier notre base de données tout en ajoutant les src et templates nécessaire dans le code pour relier la base de données au sites.

.env:

```
22  ###> doctrine/doctrine-bundle ###
23  # Format described at https://www.doctrine-project.org/projects/doctrine-dbal/en/latest/reference/configuration.html
24  # IMPORTANT: You MUST configure your server version, either here or in config/packages/doctrine.yaml
25  #
26  # DATABASE_URL="sqlite:///kernel.project_dir/var/data_%kernel.environment%.db"
27  # DATABASE_URL="mysql://app:!ChangeMe!@127.0.0.1:3306/app?serverVersion=8.0.32&charset=utf8mb4"
28  DATABASE_URL="mysql://root:@localhost:3306/my_shop_dev?serverVersion=10.4.32-MariaDB&charset=utf8mb4"
29  # DATABASE_URL="postgresql://app:!ChangeMe!@127.0.0.1:5432/app?serverVersion=16&charset=utf8"
30  ###< doctrine/doctrine-bundle ###
31
32  ###> symfony/messenger ###
33  # Choose one of the transports below
34  # MESSENGER_TRANSPORT_DSN=amqp://guest:guest@localhost:5672/%2f/messages
35  # MESSENGER_TRANSPORT_DSN=redis://localhost:6379/messages
36  MESSENGER_TRANSPORT_DSN=doctrine://default?auto_setup=0
37  ###< symfony/messenger ###
```

commande symfony pour database exemple : Doctrine:database:Create

Aussi une fois la command effectuer il est nécessaire de créer une migration à l'aide de la command : Console make:migration

puis d'effectuer la migration avec : Doctrine:migrations:migrate

Ceci permet d'envoyer à la base de donnée les modifications apporté aussi de créer automatiquement les fichiers de src et de templates concernant la modification apporté à la base de données . Les migrations permettent aussi de garder toutes les anciennes

sauvegardes des bases de données permettant de rollback sur une ancienne en cas de problème.

Le code du site est principalement divisé entre les Src et les Templates:

 assets	page5
 bin	Add webapp packages
 config	page 12
 migrations	page 9 fini
 public	page 10 fini
 src	page 12
 templates	page 12
 tests	Add webapp packages
 translations	Add webapp packages
 .env	Formulaires d'inscription et de con...

2/Src

 Controller	page 12
 Entity	page 9 fini
 Form	page 9 fini
 Repository	page 9 fini

Src est constitué de 4 principaux dossier:

Le contrôleur à pour rôle de contenir la final classe des différentes table ainsi que la route pour chaque spécifique page du site :

```
#[Route('/new', name: 'app_product_new', methods: ['GET', 'POST'])]
public function new(Request $request, EntityManagerInterface $entityManager, CategoryRepository $categoryRepository, SluggerInterface $slugger)
{
    $product = new Product();
    $form = $this->createForm(ProductType::class, $product);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $image = $form->get('image')->getData();
        if($image){
            $originalName = pathinfo($image->getClientOriginalName(),PATHINFO_FILENAME);
            $safeFileName = $slugger->slug($originalName);
            $newFileName = $safeFileName.'-'.uniqid().'.'.$image->guessExtension();
        }
    }
}
```

L'entity lui à pour but de récupérer les informations récupérées de la base de données et est constitué d'une classe et de plusieurs fonctions publiques ainsi elles sont souvent utilisées par les Controller.

Attention pour pouvoir être utilisé il faut d'abord que le controller l'appelle au début de son code.

```
use App\Repository\CategoryRepository;
use App\Entity\Category;
use App\Form\CategoryType;
use Symfony\Component\HttpFoundation\Request;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;
```

pour pouvoir utiliser chaque outils ou morceaux de code doit être appelé avant.

Le Form lui récupère les données d'entity pour les afficher stocker ;

```

public function buildForm(FormBuilderInterface $builder, array $options): void
{
    $builder
        ->add('name')
        ->add('description')
        ->add('price')
        ->add('stock')
        ->add('image', FileType::class, [
            'label' => 'Image du produit :',
            'mapped' => false,
            'required' => false,
            'constraints' => [
                new File([
                    'maxSize' => "1024k",
                    'extensions' => ['jpg', 'png', 'jpeg'],
                    'extensionsMessage' => 'Votre image doit être dans un format valide (.jpg, .png, .jpeg) !',
                    'maxSizeMessage' => 'Votre image doit faire moins de 1024k !'
                ])
            ]
        ])
        ->add('SubCategories', EntityType::class, [
            'class' => SubCategory::class,
            'choice_label' => 'name',
            'multiple' => true,
        ])
    ];
}

```

Enfin Repository contient la fonction `__construct` qui participe à la récupération des données de la base de données.

```

class AddProductHistoryRepository extends ServiceEntityRepository
{
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, AddProductHistory::class);
    }
}

```

3/Templates et assets

Name	Last commit message
..	
category	page7fini(sansreponsequestion)
home	page 12
layouts	page 12
product	page 10 fini
registration	page5
security	page5
sub_category	page8fini
user	page8fini
base.html.twig	page11fini

Name	Last commit message
..	
controllers	Add webapp packages
styles	page5
app.js	Add webapp packages
bootstrap.js	Add webapp packages
controllers.json	Add webapp packages

Assets représente le CSS du site et s'occupe du style et de la présentation du site et des images.

Templates lui contient les différentes pages HTML.twig pour chaque route de nos controller.

```
{% extends 'base.html.twig' %}

{% block title %}SubCategory{% endblock %}

{% block body %}
<div class="container">
  <h1>Sous-catégorie</h1>
  <table class="table">
    <tbody>
      <tr>
        <th>Id</th>
        <td>{{ sub_category.id }}</td>
      </tr>
      <tr>
        <th>Sous-catégorie</th>
        <td>{{ sub_category.name }}</td>
      </tr>
      <tr>
        <th>Catégorie</th>
        <td>{{ sub_category.category.name }}</td>
      </tr>
    </tbody>
  </table>
</div>
{% endblock %}
```

twig simplifie et protège contre par exemple les faille XSS notre code HTML

Aussi notre Site contient Une navbar , des flash message préparé et une méthode de pagination dans le layout dans templates.

Exemple code de pagination:

```
<nav class="navbar navbar-expand-lg bg-body-tertiary" data-bs-theme="dark">
  <div class="container">
    <a class="navbar-brand" href="#">Apollo's Shop</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarScroll" aria-controls="navbarScroll" aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarScroll">
      <ul class="navbar-nav me-auto my-2 my-lg-0 navbar-nav-scroll" style="--bs-scroll-height: 100px;">
        <li class="nav-item">
          <a class="nav-link active" aria-current="page" href="{{ path('app_home') }}">Home</a>
        </li>
        <li class="nav-item dropdown">
          <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown" aria-expanded="false">Catégories</a>
          <ul class="dropdown-menu">
            {% for category in categories %}
              <li>
                <a class="dropdown-item text-success" href="#">{{ category.name }}</a>
              </li>
              {% for subCategory in category.subCategories %}
                <li>
                  <a class="dropdown-item" href="{{ path('app_home_product_filter',{'id':subCategory.id}) }}">{{ subCategory.name }}</a>
                </li>
              {% endfor %}
            {% endfor %}
          </ul>
        </li>
      </ul>
    </div>
  </div>
</nav>
```

5/ Site d'e-commerce

Le site réalisé lors de ce TP est un site d'e-commerce dans le domaine de la vente de chaussures en ligne .

Le site possède un système de rôle obtenu lors de la connexion de l'utilisateur afin de les empêcher d'accéder au contenu du site qui ne doit pas leur être accessible comme la modification des stocks ou encore la modification des catégories .

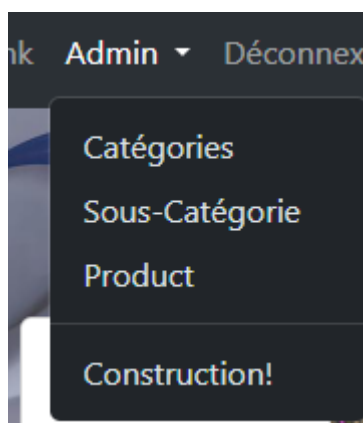
Ici nous pouvons voir la différences d'accès entre une personne qui n'est pas connecté et une personne qui l'est et qui possède le rôle ADMIN :

Hors connexion:



l'inscription ou la Connexion est demandé

Connecté avec le rôle ADMIN:



Ici la déconnexion est proposée et un nouvel onglet Admin est disponible.

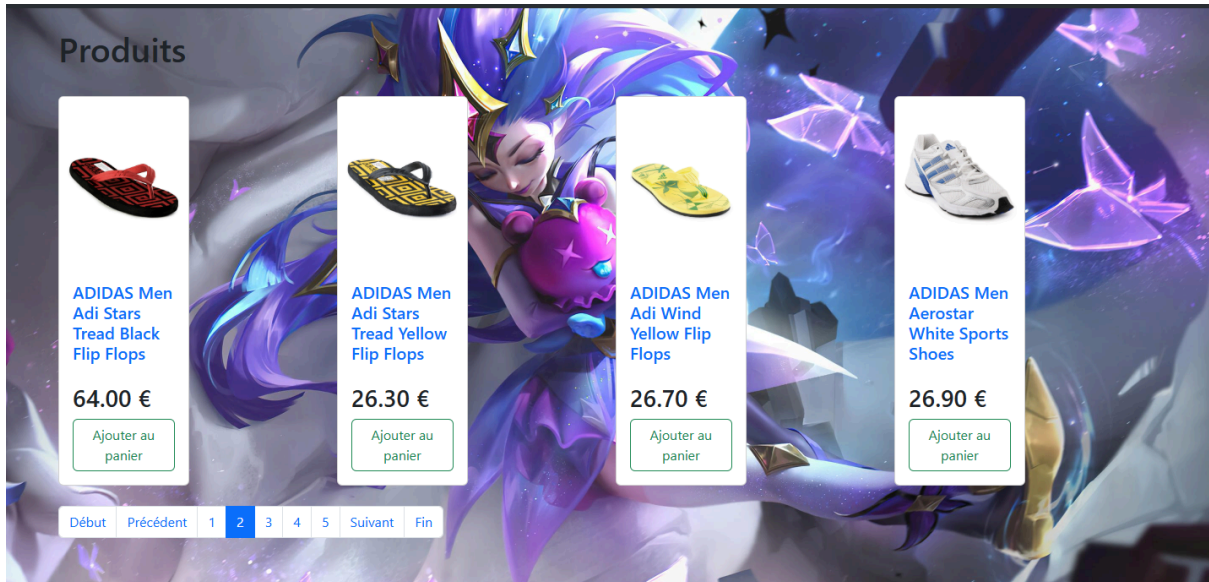
Bien évidemment on place sur la route de la page Admin pour que la page ne soit accessible que par les comptes ADMIN.

localhost:8000/admin/category

route.

L'accueil est intégré d'une présentation des produits avec une pagination pour ne pas avoir à scroll indéfiniment si trop de produit sont répertorié :

home page:



On peut voir les page en bas des cartes de produits seuls 5 sont affichés mais beaucoup plus sont disponible mais seulement montré si actuellement sur une page adjacente.

Code de la pagination:

```
{% if pageCount > 1 %}
<nav aria-label="..."
<ul class="pagination" style="display: flex; justify-content: space evenly">
  {% if previous is defined %}
  <li class="page-item">
    <a class="page-link" href="{{ path(route,query|merge({(pageParameterName):first)}})" >Début</a>
  </li>
  <li class="page-item">
    <a class="page-link" href="{{ path(route,query|merge({(pageParameterName):previous)}})" >Précédent</a>
  </li>
  {% endif %}
  {% for page in pagesInRange %}
  {% if page == current %}
    <li class="page-item active" aria-current="page">
      <a class="page-link" href="{{ path(route, query|merge({(pageParameterName):page)}})">{{ page }}</a>
    </li>
  {% else %}
    <li class="page-item" aria-current="page">
      <a class="page-link" href="{{ path(route, query|merge({(pageParameterName):page)}})">{{ page }}</a>
    </li>
  {% endif %}
  {% endfor %}

  {% if next is defined %}
  <li class="page-item">
    <a class="page-link" href="{{ path(route, query|merge({(pageParameterName):next)}})">Suivant</a>
  </li>
  <li class="page-item">
    <a class="page-link" href="{{ path(route, query|merge({(pageParameterName):last)}})">Fin</a>
  </li>
  {% endif %}
</ul>
</nav>
{% endif %}
```

Handwritten annotations: A large '1' is next to the first two lines of code. A large '2' is next to the 'Précédent' link. A large '3' is next to the active page link. A large '4' is next to the 'Suivant' link.

La première partie du code compte les pages pour voir combien de page de données il reste à afficher.

La deuxième partie du code correspond permet de revenir sur la première page ou de reculer d'une page.

La 3ème partie affiche le contenu de la page actuellement sélectionnée

enfin la dernière partie permet d'accéder à la prochaine page ou à la dernière page.

On peut voir le Path(route) a souvent apparue dans ce code afin constamment d'appeler la page sélectionnez .

Une autre fonctionnalité de ce site est le trie par catégorie et sous catégorie:

Femme

Chaussures plates

Chaussures à talons

Chaussures décontractées

Chaussures de sport

Sandales

Tongs

Homme

Chaussures de ville

Chaussures décontractées

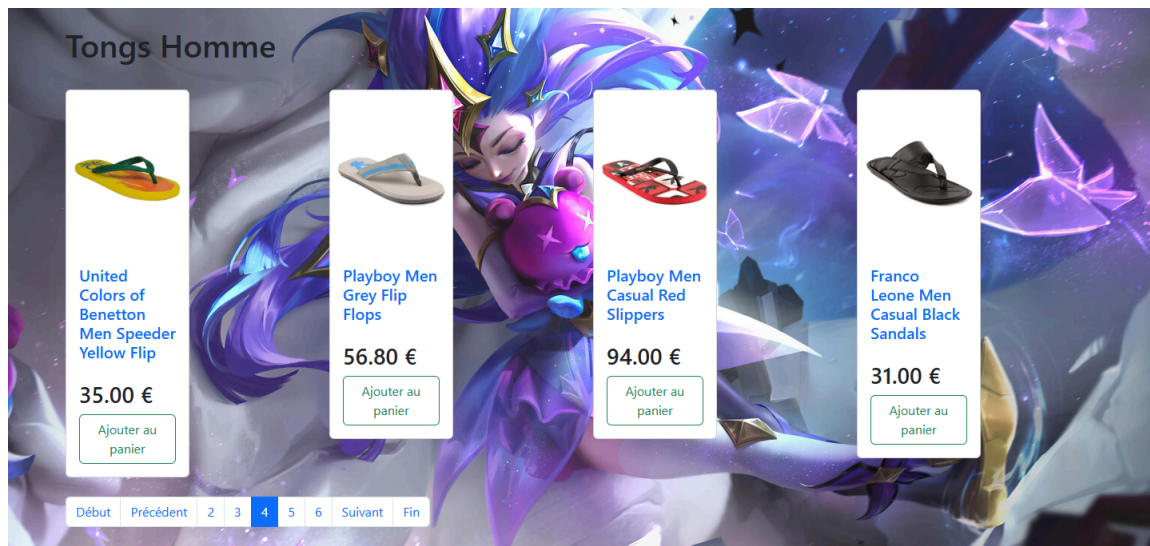
Chaussures de sport

Sandales

Tongs

Ici nous pouvons voir que la catégories et la sous catégories sont précisé (ici "Tongs Homme")

et seuls des tongs pour hommes sont affichés:



```
{% block body %}
<div class="container">
<br>
<h1>{{ subCategory.name }} {{ category.name }}</h1>
<div class="row">
{% for product in products %}
  <div class="col-md-3 mt-4">
    <div class="card" style="width: 20vh;">

      {% if product.image %}
        <a href="{{ path('app_home_product_show', {'id': product.id}) }}">
          
        {% else %}
          
        {% endif %}

        <div class="card-body">
          <a style="text-decoration: none" href="{{
            path('app_home_product_show', {'id': product.id}) }}">
            <h5 class="card-title">{{ product.name }}</h5> </a>
            <p class="card-text text-truncate">{{ product.description }}</p>
            <h3>{{ product.price }} €</h3>
            <a href="#" class="btn btn-outline-success">Ajouter au panier</a>
          </div>
        </div>
      </div>
    {% else %}
      <p>Aucun produit disponible.</p>
    {% endfor %}
  </div>
  <br>
  {{ knp_pagination_render(products, 'layouts/pagination_template.html.twig') }}
</div>
```

Ici le code du trie de la page home la première partie est la classe container utilisé par bootstrap et qui permet un style plus propre et ordonnée pour le site.

La deuxième partie est la balise qui affiche le nom de la catégorie et la sous-catégories sélectionnez (ex:Tongs Homme)

La 3ème partie récupère la carte de chaque produit avec son nom, son image et les affiche avec possibilité de l' ajouter à son panier pour les achetés.

Enfin la dernière partie appelle la pagination afin de paginer les cartes de produits pour ne pas qu'elle déborde sur le site.

Le site comporte aussi une gestion des stocks pour répertorier les différents produits qui sont disponible à la vente :

Product index					
Id	Nom	Description	Prix	Stock	actions
1	Nike Men Air Zoom Century Shoes		62.50	30	Afficher Modifier Ajout stock
2	Nike Men White Cricket Shoes		94.10	45	Afficher Modifier Ajout stock
3	Reebok Men's Ventilator Ubiq Shoe		97.30	12	Afficher Modifier Ajout stock
4	Reebok Men's Frisker LP Shoe		21.80	Stock épuisé	Afficher Modifier Ajout stock
5	Quechua Men Arpenaz Brown Sandal		76.40	18	Afficher Modifier Ajout stock
6	Nike Men's Air Impetus Shoe		20.50	37	Afficher Modifier Ajout stock
7	Nike Air Visi Sleek Shoes		46.40	17	Afficher Modifier Ajout stock
8	Puma Men Trek Navy Blue Floater		57.90	42	Afficher Modifier Ajout stock
9	Puma Men F1 Black Sandal		79.80	12	Afficher Modifier Ajout stock
10	ADIDAS Men's Black Shoe		60.20	16	Afficher Modifier Ajout stock

Des stocks peuvent être ajoutés et un historique des stocks ajouté est gardé afin de pouvoir garder l'inventaire.

Le site comportera aussi un envoie de mail de confirmation ainsi qu'un système de paiements en ligne avec Stripes qui sera ajouté aux sites ultérieurement.